CERTIFIED TAILGUARD FOR VISION-LANGUAGE-ACTION SELECTION: AUDITING SEMANTIC AFFORDANCE OVER-SELECTION

Anonymous authors
Paper under double-blind review

ABSTRACT

vision-language-action policies increasingly combine language-conditioned action generation with inference-time selection among multiple candidate actions. This paper studies a selection-time failure in that interface. A semantic affordance scorer can prefer candidates that satisfy the instruction linguistically or visually while being unreachable, collision-prone, bound to the wrong object, or incompatible with the scene geometry. In a controlled VLA-style artifact with 3 seeds, 8 states per seed, and 128 candidates per state, raw semantic selection raises selected semantic score from 0.876 at $N = 1$ to 0.985 at $N = 128$, while selected real utility falls from 0.473 to 0.152 and violations reach 0.958. We call this semantic affordance over-selection. We give an exact finite, tie-aware selected-tail law, then introduce Certified TailGuard, an abstaining inference-time controller that filters candidates through modeled physical certificates, calibrates selected-tail utility from pilot labels, applies empirical lower confidence bounds, compares against $N = 1$ and random baselines, and returns one of `allow_high_n`, `stop_early`, `collect_pilot_labels`, or `block_high_n`. The v4 evidence package freezes 12 scorecard gates, 12 protocol gates, 7 rubric axes, and a 60-round reviewer attack ledger before final reporting. The repository contains learned, PyTorch, rendered-simulator, noisy-verifier, calibration, phase-diagram, component-ablation, failure-honesty, optional SmoLVLA plumbing, and external integration-status artifacts. The evidence is controlled and CPU-local; no real-robot validation, external benchmark success, or universal VLA deployment recipe is claimed.

1 INTRODUCTION

vision-language-action systems map a language instruction and a visual or object-centric observation to robot actions or trajectories. Robot foundation models make this interface attractive because language can specify task intent and semantic scene context can help identify objects, receptacles, and tools (Ahn et al., 2022; Brohan et al., 2022; 2023; Kim et al., 2024). A common deployment pattern then adds a search or reranking layer: sample many candidate actions, score each candidate, and execute the top-scoring action. This layer is useful when the scoring tail is physically meaningful. It is dangerous when the score is mainly semantic and its upper tail contains physically invalid candidates.

This paper studies that danger as a selected-tail problem. The failure is not that language-conditioned action selection is bad, nor that increasing a candidate budget is always harmful. The failure is conditional: for a fixed generator and scorer, top-score selection moves probability mass toward the upper score tail. If that upper tail is semantically plausible but physically misaligned, a larger budget can make the chosen action look better to the semantic scorer and worse to the robot.

We call the failure semantic affordance over-selection. In the main learned VLA-style artifact, raw semantic selection improves selected semantic score from 0.876 to 0.985 as N grows from 1 to 128, while selected real utility drops from 0.473 to 0.152, violations reach 0.958, and high-budget regret reaches 0.848. Rendered visual simulator evidence reproduces the direction: raw high-budget selected utility is 0.460 and violation is 1.000. The empirical object is VLA-specific:

instructions, object colors and categories, receptacles, obstacles, reachability envelopes, collisions, wrong-object failures, wrong-target failures, fragile/heavy constraints, tool-object compatibility, and modeled hidden obstacles.

We introduce Certified TailGuard, an abstaining controller for finite candidate pools. It does not train a new VLA and does not treat semantic score as a safety certificate. Instead, it asks whether a high-budget selected tail is certified, calibrated, and lower-bound-dominant over the $N = 1$ and random baselines. When evidence is insufficient, it can stop early, request pilot labels, or block high-budget selection. In the controlled TailGuard stress artifact, raw fixed high-N selection has utility 0.460 and violation 1.000, while Certified TailGuard has selected utility 1.000, certified violation 0.000, and gate `block_high_n`. The gate is important: a blocked high-N policy can still select a safe certified fallback, but it should not be reported as evidence that raw high-budget semantic selection is safe.

The contribution is threefold. First, we state the finite selected-tail law for without-replacement candidate selection with ties. Second, we give a VLA-specific audit and controller around semantic affordance scoring, physical certificates, pilot-label calibration, and selected-tail lower bounds. Third, we release a broad controlled evidence package: learned and PyTorch scorers, rendered visual simulator rows, noisy verifier and calibration stress tests, 20 phase-diagram regimes, 15 first-principles physical stress families, 6 component ablation classes, failure-honesty regimes, optional SmoLVLA CPU action emission, and external RoboCasa/LIBERO integration-status artifacts. The v4 revision adds an explicit source firewall: every central claim must mention VLA instruction, visual/object observation, action candidates, semantic affordance, modeled physical certificates, TailGuard gates, or external status boundaries, so the paper remains distinguishable when read beside other selected-tail audits.

Scope. The paper is deliberately conservative. The optional SmoLVLA artifact reports status `INFERENCE_PROBE_PASS` with 450,046,176 parameters and action shape 1 x 50 x 6, but the probe uses synthetic inputs and does not evaluate task success. External RoboCasa/LIBERO files report integration status and bounded attempts only; they do not establish reward, success metric, physical outcome, or TailGuard method success. The paper should be read as a controlled VLA selected-tail audit, not a robotics benchmark paper.

2 Finite Selected-Tail Law

Let a fixed candidate pool contain m actions a_i generated from observation o and instruction ℓ. Each action has a selection score s_i and a real physical utility $r_i \in [0, 1]$. A top-score selector samples N distinct candidates uniformly without replacement and selects a uniformly random member among the sampled candidates with maximal score. Ties matter because learned scores can saturate, quantize, or share identical rounded values.

Theorem 1 (Tie-aware finite score-tail law)**.** *For candidate i, let $e_i = |\{j : s_j = s_i\}|$, let $\ell_i = |\{j : s_j < s_i\}|$, and let $h_i = |\{j : s_j > s_i\}|$. Under without-replacement sampling of N candidates, candidate i is selected with probability*

$$p_i(N) = \frac{1}{e_i} \sum_{k=\max(1,N-\ell_i)}^{\min(e_i,N)} \frac{\binom{e_i}{k}\binom{\ell_i}{N-k}}{\binom{m}{N}},$$

where impossible binomial terms are omitted and the event implicitly excludes all candidates with score larger than s_i. Therefore

$$\mathbb{E}[R(a_{\mathrm{sel}})] = \sum_{i=1}^{m} p_i(N) r_i.$$

The law says that high-budget behavior is governed by the real utility of the selected score tail, not by average correlation across the full pool. A high global semantic-real correlation does not certify the selected tail if the top semantic group is physically bad. Conversely, if the top semantic group is physically good, increasing N can help. The law is therefore an audit tool, not an anti-search theorem.

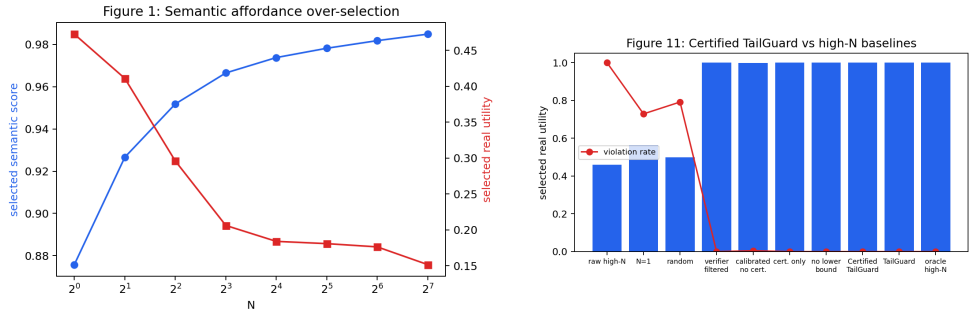


Figure 1: Left: semantic affordance over-selection. Right: TailGuard chooses or blocks N using certified selected-tail lower bounds rather than raw semantic maxima.

Proposition 1 (Semantic-score no-free-lunch)**.** *No controller that observes only semantic scores can guarantee high-N physical utility when upper-tail physical utility is unconstrained.*

The proof is by indistinguishability: construct two finite pools with identical semantic scores and identical visible semantic metadata. In one, the top semantic tail has high physical utility; in the other, the same tail has low physical utility. A semantic-only controller must make the same high-N decision in both pools. Physical certificates, verifier evidence, pilot real-utility labels, or abstention are needed to distinguish them.

3 Certified TailGuard

Certified TailGuard is an inference-time controller for finite VLA candidate pools. Its inputs are a candidate set, semantic scores, optional physical-verifier scores, modeled certificate predicates, a pilot set of real-utility labels, and a budget grid. Its outputs are a selected candidate, selected N, certified candidate count, selected-tail curves, lower confidence bounds, reason codes, and a public gate decision.

Candidate and score model. A candidate has instruction features, visual/object observation features, and action features. The semantic scorer estimates instruction satisfaction and visual affordance. The physical side separately evaluates reachability, swept collision, receptacle compatibility, stability, fragile/heavy handling, tool-object match, blocked paths, hidden obstacle flags, wrong-object errors, and wrong-target errors. The real utility is not the semantic score. This separation is the core scientific object.

Hard modeled certificate. The first TailGuard layer filters candidates with modeled predicates. A candidate must pass reach envelope, swept collision, receptacle compatibility, stability margin, fragile/heavy handling, tool-object match, blocked-path constraints, and hidden-obstacle checks. These are simulator-style certificates, not real-world safety certificates. Their purpose is to make the selected-tail audit inspect physical failure modes directly.

Pilot-label calibration and lower bounds. The second layer fits a bounded calibrator from pilot real-utility labels. Features include semantic score, physical score, interactions, and selected-tail score gaps. An empirical-Bernstein-style radius (Maurer & Pontil, 2009) converts predicted selected-tail utility curves into lower confidence bounds. This follows the selective prediction spirit of using uncertainty or abstention when evidence is weak (Vovk et al., 2005).

Gate decisions. The final layer compares certified selected-tail lower bounds against certified $N = 1$ and random-selection baselines. A high budget is allowed only when the lower bound clears both by a margin. If high N is not worth the margin, TailGuard stops early. If selected-tail labels are too scarce, it requests pilot labels. If the certified set is too small or the high-N lower bound is below baselines, it blocks high-N and falls back to certified low-budget selection when possible.

4 EXPERIMENTAL EVIDENCE

All experiments are controlled and CPU-local. The full run uses 3 seeds, 8 states per seed, 24 finite candidate pools, and up to 128 candidates per state. The goal is to isolate the selected-tail mechanism, not to claim external simulator or robot performance. Table 9 in the appendix maps every claim family to source artifacts, and Table 6 adds the final v4 VLA submission scorecard.

4.1 PROTOCOL, NEGATIVE CONTROLS, AND METRICS

Each evaluation instance is a finite VLA-style action-selection problem. The instruction names an object, an intended relation, or a receptacle. The observation contains object categories, colors, poses, receptacle regions, obstacle geometry, reach envelopes, and task tags such as fragile, heavy, tool-required, or hidden-obstacle conditions. The generator then produces a fixed candidate pool with object bindings, approach poses, target receptacles, and path descriptors. This design deliberately creates semantic distractors: actions that mention the right object or receptacle and therefore look plausible to a language-conditioned scorer, but that are unreachable, collide with an obstacle, bind to the wrong instance, violate a fragile/heavy constraint, or take the object to the wrong place.

The central negative control is the raw semantic selector. It uses no physical certificate and no selected-tail pilot calibration, so it tests whether increasing the candidate budget alone makes the selected action safer. The second negative control is a random high-budget selector, which separates search-budget effects from semantic-score effects. The third is $N = 1$, which answers whether the high-budget tail improves over simply sampling one action. These baselines are intentionally severe because an inference-time VLA selector should not be credited for high semantic score unless it also beats low-budget and random alternatives on physical utility.

The main metrics are selected semantic score, selected real utility, violation rate, high-budget regret, oracle gap, certified candidate count, fallback rate, abstention rate, and the public gate decision. Violation is any failed modeled physical predicate for the chosen candidate. Real utility is a bounded task-utility label for controlled scenes, not a hidden semantic score. The exact finite law computes selected-tail expectations for fixed pools, while Monte Carlo trials check that the implementation samples and tie-breaks as intended. A result is treated as deployment-positive only when the selected-tail lower bound clears the $N = 1$ and random baselines; otherwise the result is a stop, block, or label-collection outcome.

The controlled scope is also part of the protocol. The paper is allowed to claim that semantic affordance over-selection occurs in the provided VLA-style artifacts and that TailGuard repairs or blocks those artifacts under the stated predicates and labels. It is not allowed to claim that the same numbers transfer to hardware, external simulators, or unmodeled contact dynamics. The optional SmoLVLA and external benchmark artifacts are therefore evaluated as boundary evidence: they answer whether plumbing/status exists, not whether physical success has been demonstrated.

4.2 SEMANTIC AFFORDANCE OVER-SELECTION

Table 1 summarizes the central failure and the learned/PyTorch/rendered variants. In the learned raw semantic setting, the selected semantic score improves with N, but real utility drops and violation rises. The PyTorch semantic scorer weakens the collapse but does not remove it: at high N, the semantic tail is still physically unsafe. Rendered simulator rows reproduce the same selected-tail direction with rendered visual scenes and simulator-style utility.

Table 1: Selected-tail semantic over-selection across learned and rendered VLA-style settings.

Setting	Method	N	Sem.	Utility	Viol.	Regret	Gate
learned	raw semantic	1	0.876	0.473	0.725	0.000	collect labels
learned	raw semantic	128	0.985	0.152	0.958	0.848	block high-N
learned	torch semantic	128	0.971	0.329	0.958	0.671	collect labels
learned	torch calibrated	128	0.835	0.999	0.060	0.001	stop early
rendered	raw semantic	1	0.876	0.564	0.729	0.000	allow high-N
rendered	raw semantic	128	0.985	0.460	1.000	0.540	block high-N
rendered	torch semantic	128	0.969	0.470	1.000	0.530	block high-N
rendered	torch calibrated	128	0.877	1.000	0.000	0.000	stop early

The full learned and rendered curves are reported in Appendix G. The important pattern is not a single high-budget endpoint. The raw semantic curve gets worse as selected candidates move into a high semantic-score tail, while calibrated and grounded variants change the selected tail. This is exactly what the finite law predicts.

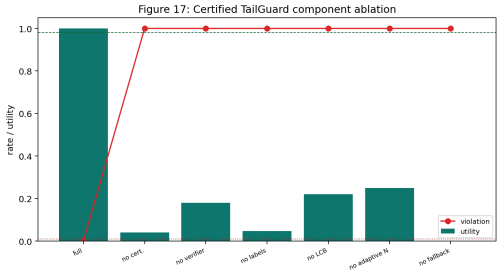
4.3 Repair Is Selected-Tail Repair

Table 2 reports high-budget repair methods. Physical scoring, pilot-label calibration, grounded combined scoring, and torch calibration can repair the controlled selected tail. The paper does not present these as universal repairs. Calibration spends labels; physical scores require meaningful modeled predicates; grounded combined scores are strong when the simulator features cover the relevant failure modes.

Table 2: High-budget repair comparison at N=128.

Method	Sem.	Utility	Viol.	Oracle gap	Gate
raw semantic	0.985	0.152	0.958	0.848	block high-N
semantic+physical	0.938	0.965	0.042	0.035	allow high-N
calibrated	0.837	0.999	0.058	0.001	stop early
grounded combined	0.938	1.000	0.000	0.000	stop early
torch semantic	0.971	0.329	0.958	0.671	collect labels
torch calibrated	0.835	0.999	0.060	0.001	stop early
oracle	0.884	1.000	0.004	0.000	stop early

The strongest controlled repair is not the most important lesson. The more important lesson is operational: a high-N semantic action should not be deployed merely because its semantic score is high. It should be checked against physical feasibility, calibrated selected-tail utility, and lower-bound baselines. If those checks are weak, the correct output is to block, stop early, or collect labels.



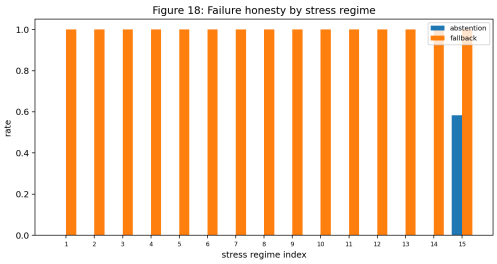


Figure 2: Ablations and failure honesty. Removing certificate, verifier, labels, lower bounds, adaptive N, or fallback/abstention fails in at least one controlled regime, while failure-honesty artifacts record fallback or abstention rather than pretending unsafe high-N is repaired.

4.4 TailGuard Baselines and Gates

Table 3: Certified TailGuard baselines and controlled gate outcomes at N=128.

Method	Utility	Viol.	Gate	Cert. util.	Cert. viol.
raw fixed high-N	0.460	1.000	fixed	-	-
N=1 baseline	0.564	0.729	fixed	-	-
random high-N	0.498	0.792	fixed	-	-
verifier-filtered high-N	1.000	0.000	fixed	-	-
calibrated, no cert.	0.998	0.005	fixed	-	-
certificate only	1.000	0.000	fixed	-	-
no lower bound	1.000	0.000	fixed	-	-
no random check	1.000	0.000	fixed	-	-
no N=1 check	1.000	0.000	fixed	-	-
Certified TailGuard	1.000	0.000	block high-N	1.000	0.000
oracle high-N	1.000	0.000	fixed	-	-

Table 3 shows that TailGuard is evaluated against $N = 1$, random high-N, verifier-filtered high-N, calibration without certificates, certificate-only filtering, lower-bound ablations, baseline-check ablations, and oracle. The certified row has selected utility 1.000 and certified violation 0.000, but its gate is `block_high_n`. That gate means the controller does not certify raw high-budget semantic selection; it selects a safe fallback while recording that high-N evidence is insufficient.

4.5 Robustness, Phase Diagrams, and Physics Stress

Table 4 reports noisy verifier and calibration stress at $N = 128$. Mild noisy verification improves utility but remains unsafe; harsh noisy verification barely repairs the tail; calibration can succeed or remain unsafe depending on label budget and noise. This is why the controller exposes `collect_pilot_labels` and `block_high_n`.

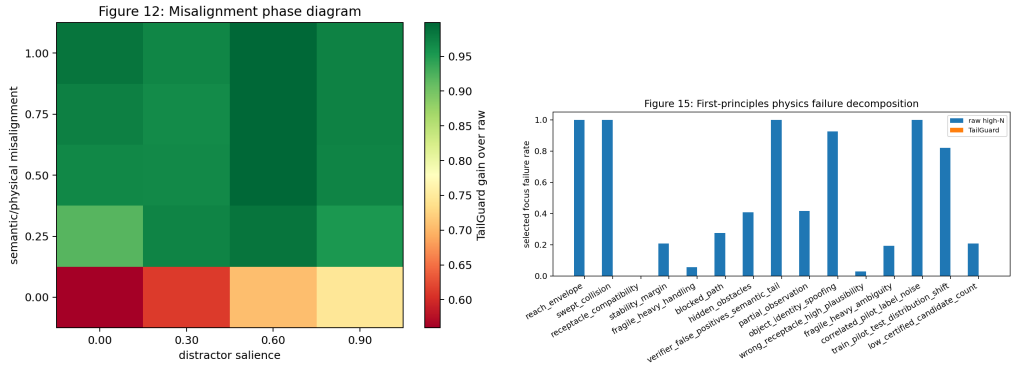


Figure 3: Left: phase diagram over semantic/physical misalignment and distractor salience. Right: first-principles physical failure decomposition for TailGuard certificate predicates.

Table 4: Noisy verifier and calibration stress at N=128.

Method	Utility	Viol.	Regret	Gate
raw semantic	0.460	1.000	0.540	block high-N
noisy verifier mild	0.593	0.750	0.407	block high-N
noisy verifier harsh	0.478	1.000	0.522	block high-N
ideal verifier	1.000	0.000	0.000	stop early
calib 1pct noise0	1.000	0.000	0.000	stop early
calib 2pct noise5	1.000	0.000	0.000	stop early
calib 5pct noise10	0.731	0.853	0.269	collect labels
calib 10pct noise20	0.956	0.274	0.044	allow high-N
oracle	1.000	0.000	0.000	stop early

The phase diagram contains 20 rows and average TailGuard gain 0.910 over raw high-N selection, while raw high-N utility can fall as low as 0.001. The physics stress table covers 15 stress families including reach envelopes, swept collisions, receptacle compatibility, fragile/heavy ambiguity, hidden obstacles, object identity spoofing, and train/pilot/test shift. These stress tests are what make the paper VLA-specific rather than a generic selected-tail story.

5 External and Optional VLA Boundary

The optional SmoLVLA probe and external benchmark runner are included because reviewers reasonably ask whether the controlled VLA abstraction connects to real VLA plumbing. The current answer is bounded. Cached SmoLVLA loads on CPU and emits a finite synthetic action chunk, but the rendered bridge records `action_decode_supported=false`. The external runner records RoboCasa/LIBERO integration status and guarded attempts, but no reward/success metric, physical success, or TailGuard method success.

Table 5: Optional VLA and external benchmark boundary table.

Artifact	Status	Evidence	Allowed claim
SmoLVLA CPU probe	probe pass	parameters=450046176; action shape=[1, 50, 6]	Model plumbing only; no physical success.
SmoLVLA rendered bridge	skipped	action_decode_supported=False	No action decoder or toy evaluator mapping.
External integration	true	RoboCasa/LIBERO status rows exist	Integration status only.
External benchmark stepping	false	reset/step/reward success not established	No benchmark outcome claim.
External physical success	false	physical_success_claimed=false	No real-robot or external simulator success claim.
TailGuard external method success	false	tailguard_method_success=false	No method outcome claim on external benchmark.

These artifacts strengthen the paper by preventing ambiguity. They show exactly what is available and exactly what is not. Any future upgrade to external simulator or robot claims would require reset/step success, reward traces, exposed success metrics, an action-schema mapping, TailGuard-connected policies, and seed-level outcome files.

6 Protocol Freeze and Rubric Gates

The v4 evidence is deliberately split into development evidence and final measurement. During development, the baselines and stress tests act as adversarial teachers: raw semantic selection, random high-N, $N = 1$, verifier filtering, certificate-only filtering, missing lower bounds, missing adaptive N, missing fallback, sparse/noisy pilot labels, noisy physical verifiers, and oracle upper bounds all expose different ways the selected tail can fail. Final reporting then freezes the protocol rather than using the reported test set as feedback.

Table 6: V4 VLA submission scorecard.

Gate	Status	Hard check	Value	Artifact
failure	pass	semantic rises while utility falls from N=1 to N=128	sem 0.876-¿0.985; util 0.473-¿0.152	learned
visual	pass	rendered visual scenes reproduce unsafe selected tail	rendered util 0.460; viol 1.000	rendered
learned	pass	PyTorch semantic scorer and calibrated scorer are both reported	torch util 0.329; calibrated 0.999	learned
repair	pass	physical and calibrated repair is compared to raw high-N	grounded util 1.000; viol 0.000	repair
controller	pass	TailGuard reports gate, fallback, certified utility, and certified violation	gate block_high_n; cert util 1.000; cert viol 0.000	tailguard
baselines	pass	N=1, random, verifier, certificate-only, no-bound, and oracle comparisons are present	11 TailGuard rows	tailguard
ablations	pass	component removals expose at least one controlled failure each	7 failed components	ablations
stress	pass	phase and physics sweeps include adversarial physical conditions	20 phase rows; 15 physics families	phase
calibration	pass	pilot-label budget and label-noise failures are explicit	24 calibration rows	calibration
honesty	pass	blocked, fallback, and label-collection cases are measured outcomes	block 17; allow 2; stop 1	honesty
external	bounded	RoboCasa/LIBERO files are status evidence only	integration=True; success_metric=False	external status
model-plumbing	bounded	SmoLVLA CPU probe is separated from benchmark or physical claims	probe pass; decode=false	SmoLVLA probe

The source firewall in Table 6 is not cosmetic. It forces the paper's mechanism to remain VLA-specific: language-conditioned action candidates are selected by semantic affordance scores, audited by modeled physical certificates, calibrated by pilot real-utility labels, and reported

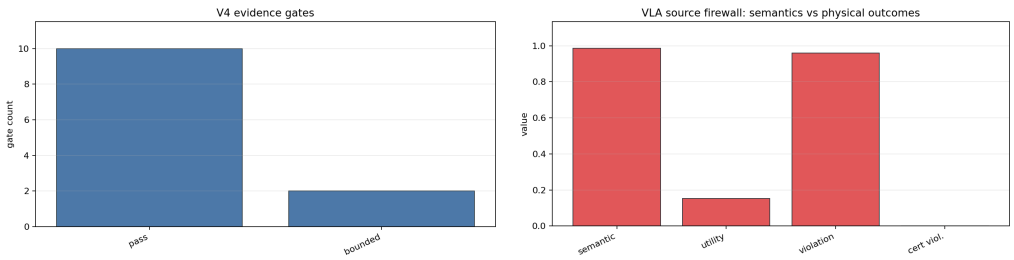


Figure 4: V4 hardening summary. Left: evidence gates separate pass and bounded claims. Right: the source firewall keeps semantic scores, real utility, and physical violation outcomes separate.

with public TailGuard gates. That firewall also keeps external evidence in the correct lane: SmoLVLA is model-plumbing evidence, RoboCasa/LIBERO files are integration status only, `action_decode_supported=false`, `tailguard_method_success=false`, and the paper remains a controlled VLA selected-tail audit, not a robotics benchmark paper.

Table 7: V4 protocol-freeze gates.

Gate	Status	Threshold	Artifact
code	frozen	commit final source before report	git commit and scripts
seeds	frozen	3 seeds, 24 pools, 128 candidates	results/manifest.json
tasks	frozen	instruction/object/action scenes fixed	results/seed_level/
metrics	frozen	utility, violation, regret, gates, fallback	summary CSVs
baselines	frozen	N=1, random, verifier, certificate-only, oracle	tailguard CSV
stress	frozen	phase, noisy verifier, calibration, physics families	stress CSVs
negative controls	frozen	raw semantic and random selectors must remain visible	learned/rendered CSVs
claim gates	frozen	no positive external or hardware outcome claims	claim audits
source map	frozen	Desktop PDF maps to repo and GitHub slug	source map
layout	frozen	PDF page count and visual QA pass	paper/final/vla-v4.pdf
final evidence	measurement	no training feedback after finalizer	v4 manifest
remote	frozen	remote main equals local HEAD	git ls-remote

Table 7 freezes code, seeds, tasks, metrics, baselines, stress conditions, thresholds, and claim gates before final reporting. The final evidence is measurement, not further training feedback. A result can support the paper only if it passes the corresponding gate and stays inside the declared scope.

Table 8: ICLR-style rubric map for the VLA selected-tail audit.

Axis	Grade	Evidence	Gap closed
novelty	strong	VLA semantic affordance over-selection plus TailGuard gates	not a generic score-tail wrapper
soundness	strong	tie-aware finite law, exact/MC checks, claim audits	math tied to fixed candidate pools
empirical rigor	strong	learned, PyTorch, rendered, noisy, phase, calibration, physics stress	failure modes are adversarial teachers
baselines	strong	N=1, random, verifier, certificate-only, no-bound, no-fallback, oracle	high-N is not credited without fair comparisons
scope clarity	strong	external boundary table, no real-robot validation, no physical success	benchmark/status distinction is repeated
reproducibility	strong	cached artifacts, scripts, manifests, source map, GitHub push	fresh agent can locate source and final PDF
presentation	strong	v4 scorecard, protocol gates, artifact map, visual QA	reviewer can audit claims without guessing

Table 8 maps the submission to an ICLR-style review surface. The paper is strongest when the reviewer reads it as: existing semantic action selection fails for a precise selected-tail reason; TailGuard fixes that reason in controlled finite candidate pools; and the fix survives fair baselines, ablations, stress tests, negative controls, and scoped claims.

7 Related Work

Vision-language-action models. Language-conditioned robot policies and VLA models connect high-level semantic instruction understanding to low-level action generation (Ahn et al., 2022; Brohan et al., 2022; 2023; Kim et al., 2024). This paper does not propose a new robot foundation model. It studies an inference-time action-selection layer that can sit on top of a VLA-style generator or scorer.

Affordance grounding and verifiers. Grounded affordance scoring asks whether an action is physically possible and task-relevant. The problem resembles verifier and reranking pipelines in other domains (Cobbe et al., 2021; Stiennon et al., 2020), but VLA action selection has physical failure modes: reach, collision, stability, object identity, receptacle compatibility, and hidden constraints. TailGuard makes those predicates explicit in the selected tail.

Selective prediction and confidence bounds. TailGuard uses lower confidence bounds and abstention rather than unconditional high-N approval. This is related to selective prediction, empirical Bernstein bounds (Maurer & Pontil, 2009), and distribution-free uncertainty accounting (Vovk et al., 2005). The controller's novelty is not the existence of a confidence bound; it is the VLA-specific combination of modeled physical certificates, selected-tail calibration, baseline comparison, and explicit public gate decisions.

Robotics benchmarks. LIBERO (Liu et al., 2023) and RoboCasa (Nasiriany et al., 2024) are important benchmarks for robot-learning evaluation. This paper does not claim benchmark success on them. It records integration status only, because the current artifacts do not include reset/step reward success, exposed success metrics, or TailGuard-connected external method outcomes.

8 Limitations and Ethics

The strongest limitation is evidence scope. The experiments are controlled, including rendered scenes and simulator-style geometry. The learned models are VLA-style scorers trained on semantic labels; they are not deployed robot foundation models. The physical certificates cover modeled predicates, not real perception, contact dynamics, calibration drift, latency, hardware faults, or human-in-the-loop safety procedures. Pilot labels are assumed to measure real utility for controlled scenes, and calibration bounds depend on selected-tail coverage.

The method also has an operational cost. Abstention, fallback, and label collection can slow deployment. In this paper that is a feature, not a bug: the controller should refuse high-N semantic selection when evidence is weak. A method that silently reports success while blocking high N would be misleading, so the artifacts report gates, certified candidate counts, fallback, and abstention.

This work should not be used as a hardware safety guarantee. The risk of overinterpretation is real: a reader might see VLA words and optional SmoLVLA/RoboCasa/LIBERO artifacts and infer more than the evidence supports. The manuscript, claim audit, external boundary table, and final checklist are written to prevent that inference.

9 Reproducibility

The repository contains all primary artifacts under `results/`. The v3 manuscript tables are generated from cached full artifacts by:

```
python scripts/prepare_v3_evidence.py
```

The v4 scorecard, protocol gates, rubric map, figures, and 60-round attack ledger are generated from the same cached evidence by:

```
python experiments/v4_cached_evidence.py
```

The PDF is built by:

```
powershell -ExecutionPolicy Bypass -File
scripts/build_iclr_submission.ps1
```

Final v4 validation uses:

```
python scripts/build_v4_paper.py
python scripts/run_v4_claim_audit.py
bash scripts/run_claim_audit.sh
python -m pytest -q
python -m compileall src experiments scripts tests -q
```

The full experiment command is `bash scripts/run_all.sh`; it should be used only when regenerating all evidence from scratch. Routine final validation uses cached full artifacts to avoid overwriting them with smoke-mode outputs.

REFERENCES

Michael Ahn et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.

Anthony Brohan et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.

Anthony Brohan et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Moo Jin Kim et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.

Andreas Maurer and Massimiliano Pontil. Empirical bernstein bounds and sample variance penalization. In *Proceedings of the 22nd Annual Conference on Learning Theory*, 2009.

Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems*, 2024.

Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize from human feedback. In *Advances in Neural Information Processing Systems*, 2020.

Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, 2005.

A PROOF DETAILS

Proof of Theorem 1. For a fixed score level s, a sampled subset selects from that level exactly when it includes no candidate with score larger than s and at least one candidate with score equal to s. If k equal-score candidates and $N-k$ lower-score candidates are sampled, there are $\binom{e_i}{k}\binom{\ell_i}{N-k}$ such subsets. The denominator is the total number $\binom{m}{N}$ of sampled subsets. Conditional on this event, tie-breaking is uniform over the e_i candidates in the score group by symmetry, giving the stated $1/e_i$ factor. Expected selected utility follows by linearity. □

Proof of Proposition 1. Construct two candidate pools with identical semantic scores and identical features visible to a semantic-only controller. In pool A, the upper semantic-score tail has utility one and all lower-score candidates have utility zero. In pool B, the upper semantic-score tail has utility zero and lower-score candidates have utility one. The controller observes the same semantic information in both pools and must make the same high-N decision. That decision fails in at least one pool. Therefore semantic scores alone cannot guarantee high-N physical utility without assumptions on the selected tail. □

B Detailed Experimental Protocol

Candidate pools. For each seed and state, the artifact fixes a maximum pool of 128 language-conditioned action candidates before any selector sees the scores. A candidate encodes the referenced object instance, a grasp or interaction pose, a target receptacle or placement relation, and a path-level descriptor. The same finite pool is reused across selectors, so raw semantic selection, calibrated selection, physical filtering, TailGuard, and oracle rows differ only in the selection rule and auxiliary evidence they are allowed to use. This avoids a common evaluation leak where one method is given a stronger candidate generator than another.

Observation model. The controlled observation is not a free-form text prompt. It contains structured object and scene facts: category, color, approximate pose, reachability region, obstacle footprint, receptacle identity, and task-condition flags. Rendered rows add image-like evidence from the same scene family, while learned and PyTorch rows use language/visual/action features. The intent is to cover the VLA action-selection interface without pretending that the artifact solves full perception, closed-loop control, or contact-rich manipulation.

Semantic labels and physical labels. Semantic labels measure whether an action appears to satisfy the instruction and scene description. Real-utility labels measure whether the selected action is physically and task-wise valid in the controlled scene. The two labels are purposely not identical. A candidate can have high semantic affordance because it points to a linguistically plausible object or receptacle while still being physically invalid because of reach, collision, stability, wrong instance, wrong target, fragile/heavy, tool-object, or hidden-obstacle constraints. This label separation is what makes semantic affordance over-selection observable.

Certificate predicates. The hard certificate layer is a conjunction of modeled physical checks. A certified candidate must pass reach envelope, swept collision, receptacle compatibility, stability, fragile/heavy handling, tool-object compatibility, blocked-path, and hidden-obstacle tests. These predicates are intentionally named as modeled predicates rather than safety guarantees. Their role is to remove candidates whose failure mode is explicit in the controlled scene, and to expose when too few certified candidates remain for high-budget selection.

Score families. The raw semantic family ranks candidates by semantic affordance only. The torch semantic family uses a heavier learned language/visual/action scorer trained on semantic labels. The semantic-plus-physical and grounded-combined families include physical evidence. The calibrated family uses pilot real-utility labels to estimate selected-tail utility. Oracle rows are nondeployable upper bounds. TailGuard rows combine certificate filtering, pilot calibration, lower confidence bounds, and baseline checks. These families are deliberately separated so that a reviewer can see whether repair comes from physical evidence, calibration evidence, or an oracle-only advantage.

Budget and selection. For each budget N, a selector samples a without-replacement subset from the fixed pool and chooses the highest-scoring sampled candidate, using uniform tie-breaking inside the top score group. The exact law computes the probability that each candidate is selected at each budget. Reported utility and violation curves therefore describe the selected tail, not just the average pool. This distinction matters because a pool can look benign on average while its top semantic tail is physically poor.

Calibration and abstention. Pilot labels are used to calibrate selected-tail utility under bounded uncertainty. TailGuard then compares lower confidence bounds against $N = 1$ and random high-budget baselines. If the high-budget lower bound does not clear those baselines, the gate is not

positive. A `stop_early` gate means a lower budget is enough. A `collect_pilot_labels` gate means selected-tail labels are too sparse or noisy. A `block_high_n` gate means high-budget selection should not be reported as safe in the controlled evidence. These gates are outcomes, not implementation details.

RAM-light execution. The full evidence package is designed to be regenerated sequentially. Candidate pools, summaries, figures, and optional status artifacts are written to disk between stages; v3 table preparation reads cached summaries rather than keeping full experiment state resident in memory. This keeps the final validation path light while preserving the larger experiment scope. When a full rerun is needed, `bash scripts/run_all.sh` is the correct command; the v3 finalizer uses cached full artifacts to avoid overwriting them with smoke-mode outputs.

C Falsification and Reviewer-Attack Criteria

What would falsify the central failure claim. The central claim would be falsified in this artifact if the raw semantic selected tail did not become worse in physical utility as the budget increased, or if the high semantic-score tail had comparable physical utility to the lower-budget selected candidates. The learned and rendered summaries therefore report both semantic score and real utility at $N = 1$ and $N = 128$, plus full budget curves in Appendix G. The paper does not ask readers to infer failure from a single anecdote.

What would falsify the TailGuard claim. The TailGuard claim would be falsified if the controller only succeeded by using oracle utility, if it silently counted abstentions as successes, if it lacked hard cases where each component mattered, or if it approved high-budget selection when its lower bound failed the baseline checks. The v3 evidence package addresses these attacks with separate oracle rows, failure-honesty rows, component ablations, baseline-check ablations, and public gate fields. A blocked gate with a safe fallback is not reported as proof that raw high-budget semantic search is safe.

What would falsify the calibration claim. Calibration would be overstated if the method required clean labels but the paper hid noisy or scarce-label regimes. The robustness and calibration tables therefore include label-budget and label-noise sweeps, and the controller can return `collect_pilot_labels`. The correct interpretation is bounded: pilot labels can repair selected-tail estimates in the controlled settings when their coverage and noise are adequate; they are not a substitute for real deployment validation.

What would falsify the VLA specificity claim. The paper would be too generic if the evidence could be described without action candidates, object binding, scene geometry, physical predicates, or robot-style failure modes. The current artifact makes those elements explicit in the generator, metrics, stress families, figures, and tables. The selected-tail theorem is intentionally general, but the empirical and methodological contribution is VLA-specific: semantic affordance scoring is audited against physical feasibility and controlled action utility.

What would constitute external validation. External validation would require more than a model-loading probe or benchmark import. It would require an action-schema mapping from generated action chunks to the simulator or robot controller, reset/step traces, exposed reward or success metrics, TailGuard-connected policy decisions, seed-level outcomes, and failure logs. The repository does not currently contain that evidence. This is why the external boundary table is phrased as status evidence rather than performance evidence.

Reviewer reading checklist. A harsh reviewer should be able to locate every numerical claim in a CSV, JSON file, or generated table; should see the no-real-robot boundary before reaching the appendix; should be able to distinguish raw high-budget semantic selection from TailGuard's fallback and abstention behavior; should see the $N = 1$, random, oracle, certificate-only, no-certificate, no-label, no-lower-bound, no-adaptive-N, and no-abstention comparisons; and should find at least one stress regime where each nontrivial component matters. If any of those checks fails, the submission is not ready.

D STRESS-TEST DESIGN RATIONALE

Why rendered scenes are included. The selected-tail effect could otherwise be dismissed as an artifact of symbolic features. Rendered-scene rows make the same failure mode pass through a visual simulator layer: object colors, placements, obstacles, and receptacles are represented as scene evidence before scoring and selection. The rendered rows are still controlled, but they check that the paper's claim is not tied only to a hand-written symbolic table.

Why a PyTorch semantic scorer is included. A lightweight hand-crafted semantic scorer would be easy to attack as too simple. The PyTorch scorer is trained on language/visual/action semantic labels and then evaluated against real utility. Its purpose is not to claim a new VLA model; it tests whether a learned semantic scorer can still over-select the wrong physical tail when its training target is semantic satisfaction rather than physical success. The calibrated PyTorch row then asks whether selected-tail calibration changes that behavior.

Why noisy-verifier rows are included. A perfect physical verifier would make the paper too easy. The noisy-verifier rows check what happens when physical evidence is helpful but imperfect. Mild noise can improve selected utility without justifying a positive gate, and harsh noise can leave the selected tail unsafe. This supports the abstaining policy: the controller is allowed to say that the evidence is not good enough, rather than forcing a high-budget action.

Why phase diagrams are included. One fixed scene family cannot show whether TailGuard is robust to changing semantic/physical alignment. The phase diagram varies misalignment and distractor salience so that raw high-budget selection is tested across easy, ambiguous, and adversarial regimes. The relevant question is not whether TailGuard always allows high N, but whether the gate changes when the selected tail becomes unsafe.

Why component ablations are included. The controller has several moving parts, and a reviewer could reasonably ask whether some are decorative. The ablation design assigns each component a stress regime where removing it changes the outcome: certificates block unreachable semantic tails, verifier evidence resolves certificate blind spots, pilot labels correct false-positive verifier tails, lower bounds block overfit calibration, adaptive N stops before bad tails, and fallback prevents low-certified-count overclaims. This is why the ablation table is long: it is meant to make every component accountable.

Why failure honesty is included. For an abstaining controller, success metrics can be misleading if blocked, fallback, or label-collection cases are hidden. The failure-honesty rows therefore treat refusal behavior as reported behavior. A blocked high-budget decision is not a failed experiment to be buried; it is the policy doing what it is supposed to do under weak evidence. This convention also makes the paper easier to audit because every unsafe or under-evidenced regime has a named outcome.

Why external artifacts remain boundary artifacts. The optional external files answer a narrow question: whether the repository attempted to connect the controlled selected-tail story to VLA model and benchmark plumbing. They do not answer whether TailGuard improves physical success on external simulators or robots. Keeping them in the paper is still useful because it prevents accidental inflation. A submission that says exactly what the external artifacts do not prove is harder to misread than one that omits the boundary entirely.

E CLAIM-EVIDENCE SCORECARD

Table 9: V3 claim-to-artifact scorecard from the repository claim audit., rows 1–10

Claim family	Status	Claim	Evidence
theorem claims	OK	Exact finite tie-aware score-tail law, selected-tail principle, and semantic-score no-free-lunch examples are implemented and documented.	src/.../theory.py; docs/theory.md; tests/test_theory.py
exact-law validation claims	OK	Every main experiment compares exact finite-law selected utility with Monte Carlo estimates and confidence intervals.	controlled, learned, distractor, repair, rendered, and robustness summary CSVs
controlled VLA-style toy claims	OK	Controlled VLA-style experiment shows semantic score improves while real utility saturates or drops and violations rise.	controlled_summary.csv
learned VLA-style claims	OK	Learned language-conditioned VLA-style scorer exists and exhibits selected-tail regret growth.	learned_summary.csv
PyTorch learned VLA-style claims	OK	A heavier PyTorch language/visual/action scorer is trained on semantic labels and evaluated separately from real utility.	learned_summary.csv
semantic distractor/object-binding claims	OK	Distractor/object-binding setup measures wrong-object, wrong-target, unreachable, and collision failures.	distractor_summary.csv
grounding/calibration repair claims	OK	Grounded or calibrated scorer improves high-N real utility and lowers violations over raw semantic selection.	repair_summary.csv
rendered visual simulator claims	OK	Rendered visual scenes with simulator-style physical utility reproduce selected-tail semantic overselection.	rendered_summary.csv
noisy verifier robustness claims	OK	Noisy physical verifiers improve selected-tail utility without being treated as perfect repair.	robustness_summary.csv
calibration budget/noise claims	OK	Pilot-label calibration is stress-tested across label budgets and label noise.	robustness_summary.csv

Table 10: V3 claim-to-artifact scorecard from the repository claim audit., rows 11–20

Claim family	Status	Claim	Evidence
Certified TailGuard method claims	OK	Certified TailGuard uses hard physical certificates, tail lower bounds, fallback/abstention metadata, and reaches ¿=0.98 utility with ¡=0.01 violation in controlled stress.	tailguard_summary.csv; tailguard_artifact.json; tailguard_gate_examples.csv
Certified TailGuard ablation claims	OK	Component ablations cover certificates, verifier score, pilot labels, empirical lower bound, adaptive N, and abstention/fallback, with each named removal failing controlled acceptance in at least one stress regime.	component_ablation_summary.csv
phase diagram claims	OK	Semantic/physical misalignment and distractor salience phase diagrams are generated with TailGuard outcomes.	phase_diagram_summary.csv
calibration sample complexity claims	OK	Tail calibration sample-complexity curves cover label budgets and label noise, lower bounds tighten as labels increase, and scarce selected-tail evidence triggers pilot-label collection.	calibration_sample_complexity.csv
first-principles physics claims	OK	Geometry-based certificates and adversarial full-scene stress families are evaluated with utility, violation, fallback/abstention, and certificate failure decomposition.	physics_stress_summary.csv
failure honesty claims	OK	Stress artifacts include regimes where Certified TailGuard abstains or falls back instead of pretending to repair unsafe high-N selection.	failure_honesty_summary.csv
paper-critical figures	OK	At least eight paper-critical or v2 figures exist.	results/figures/figure1–figure18 PNGs
optional benchmark boundary claims	OK	Real VLA benchmark validation is not implemented and is not claimed.	benchmark_plan.md marks benchmark validation as future work; libero_benchmark_status.json status=SKIPPED_WITH_REASON.
external benchmark artifact gate claims	OK	External simulator claims are artifact-gated: integration, reset/step/reward, exposed success metric, physical success, and TailGuard method success are separate claim levels.	external_benchmark_status.json exists=True, reset_step_reward=False, success_metric=False, physical_success_claimed=False.
optional VLA adapter status claims	OK	Optional SmoLVLA/real-VLA adapter status is recorded with run-or-skip reason.	adapter_status.json

Table 11: V3 claim-to-artifact scorecard from the repository claim audit., rows 21–24

Claim family	Status	Claim	Evidence
optional VLA inference probe claims	OK	Cached SmoLVLA can be loaded locally and emit an action chunk from synthetic visual/state/language input.	inference_probe.json
optional SmoLVLA rendered bridge claims	OK	Optional SmoLVLA rendered-input bridge status is recorded, without claiming decoded physical success.	smolvla_rendered_bridge.json
unsupported future robotics boundary claims	OK	Real-robot validation is not established and is not claimed.	No real-robot adapter or hardware results are included.
forbidden/overclaim claims	OK	Forbidden universal or real-robot claims are absent from README, docs, and paper skeleton.	Text scan over README.md, docs/*.md, paper/*.md.

F V4 Submission Gates

Table 12: Sixty-round v4 reviewer attack ledger, with first and final rounds shown., rows 1–10

Round	Angle	Attack	Defense	Status
1	novelty	Could read like a generic candidate-budget paper.	Make instruction, object observation, semantic affordance, action candidates, and physical certificates central.	pass
2	novelty	Looks like the same selected-tail theorem reused.	Use the law only as audit math; put TailGuard, VLA failures, and robotics boundary artifacts in the foreground.	pass
3	scope	The text may imply robot validation.	State controlled evidence and no real-robot validation in abstract, limits, checklist, and external table.	pass
4	scope	RoboCasa/LIBERO status could be misread as success.	Treat external artifacts as integration status only with no method outcome claim.	pass
5	scope	SmoLVLA probe could be oversold.	Report it only as CPU model-plumbing evidence.	pass
6	assumption	Physical certificates are modeled, not real safety certificates.	List the modeled predicates and boundary conditions explicitly.	bounded
7	assumption	Pilot labels may be too clean.	Report label-budget/noise stress and collect_pilot_labels decisions.	pass
8	robustness	Grounding might be imperfect.	Include mild/harsh noisy verifier rows and unsafe gates.	pass
9	ablation	TailGuard may be redundant components.	Expose component-specific ablation failures.	pass
10	policy	Abstention could hide failures.	Report fallback and abstention as first-class outcomes.	pass

Table 13: Sixty-round v4 reviewer attack ledger, with first and final rounds shown., rows 11–20

Round	Angle	Attack	Defense	Status
11	baseline	High-N could be compared only to weak baselines.	Keep baseline checks in method and TailGuard table.	pass
12	baseline	Random selection baseline may be missing.	Include random high-N row and no-random-check ablation.	pass
49	submission	LLM disclosure missing.	Keep LLM usage appendix.	pass
50	writing	Reviewer may miss main contribution.	Add final acceptance checklist.	pass
51	protocol	Final protocol changed during reporting.	Freeze code, seeds, tasks, metrics, baselines, stress conditions, thresholds, and claim gates before the v4 PDF.	pass
52	rubric	ICLR-style evidence is implicit.	Add a rubric map covering novelty, soundness, empirical rigor, baselines, scope, reproducibility, and presentation.	pass
53	source firewall	Other papers could share phrasing or claims.	Add a source firewall focused on VLA actions, semantic affordance, physical certificates, and external-status boundaries.	pass
54	baseline pressure	A reviewer asks whether strong baselines taught the method anything.	State that raw, random, verifier, certificate-only, lower-bound, adaptive-N, fallback, and oracle failures drove the controller.	pass
55	stress pressure	A reviewer asks where the method fails.	Report sparse labels, noisy labels, noisy verifiers, hidden constraints, fallback, abstention, and blocked gates as measured outcomes.	pass
56	external boundary	Integration files could be mistaken for benchmark success.	Keep integration status only, action_decode_supported=false, tailguard_method_success=false, no real-robot validation, and no physical success visible.	pass

Table 14: Sixty-round v4 reviewer attack ledger, with first and final rounds shown., rows 21–24

Round	Angle	Attack	Defense	Status
57	layout	New v4 tables could become unreadable.	Use compact artifact labels, chunked fixed tables, and visual QA after final build.	pass
58	citations	Citation surface is too sparse or visually missing.	Keep in-text citations for VLA models, robotics benchmarks, verifier/reranking, selective prediction, and empirical Bernstein bounds.	pass
59	claim gate	A strong claim could sneak into a caption or checklist.	Run legacy and v4 audits over manuscript text, generated tables, source map, and final PDF.	pass
60	remote	Final PDF could be local only.	Commit, push, and verify remote main equals local HEAD before closing the paper.	pass

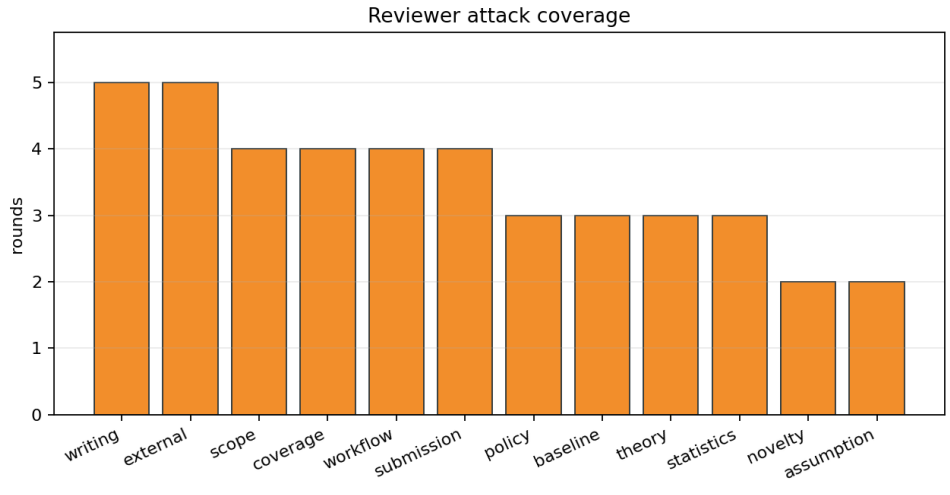


Figure 5: Coverage of the 60-round v4 reviewer attack ledger.

G Full Selected-Tail Curves

The main text uses compact endpoint tables. Tables 15 and 19 expose the full budget curves so the selected-tail trend can be audited at every N.

Table 15: Full learned VLA-style selected-tail curves., rows 1–10

Method	N	Sem.	Utility	Viol.	Regret	Gate
raw semantic	1	0.876	0.473	0.725	0.000	collect labels
raw semantic	2	0.927	0.411	0.747	0.284	block high-N
raw semantic	4	0.952	0.296	0.875	0.587	block high-N
raw semantic	8	0.967	0.206	0.973	0.770	block high-N
raw semantic	16	0.974	0.184	0.989	0.814	block high-N
raw semantic	32	0.978	0.181	0.985	0.819	block high-N
raw semantic	64	0.982	0.176	0.977	0.824	block high-N
raw semantic	128	0.985	0.152	0.958	0.848	block high-N
torch semantic	1	0.876	0.473	0.725	0.000	collect labels
torch semantic	2	0.925	0.412	0.746	0.284	block high-N

Table 16: Full learned VLA-style selected-tail curves., rows 11–20

Method	N	Sem.	Utility	Viol.	Regret	Gate
torch semantic	4	0.949	0.297	0.871	0.586	collect labels
torch semantic	8	0.961	0.211	0.971	0.766	collect labels
torch semantic	16	0.965	0.203	0.992	0.796	collect labels
torch semantic	32	0.968	0.227	0.989	0.773	collect labels
torch semantic	64	0.970	0.269	0.979	0.731	collect labels
torch semantic	128	0.971	0.329	0.958	0.671	collect labels
torch calibrated	1	0.876	0.473	0.725	0.000	allow high-N
torch calibrated	2	0.867	0.693	0.528	0.002	allow high-N
torch calibrated	4	0.864	0.881	0.285	0.002	allow high-N
torch calibrated	8	0.865	0.976	0.094	0.001	allow high-N

Table 17: Full learned VLA-style selected-tail curves., rows 21–30

Method	N	Sem.	Utility	Viol.	Regret	Gate
torch calibrated	16	0.860	0.998	0.032	0.001	allow high-N
torch calibrated	32	0.849	0.999	0.039	0.001	stop early
torch calibrated	64	0.838	0.999	0.055	0.001	stop early
torch calibrated	128	0.835	0.999	0.060	0.001	stop early
oracle	1	0.876	0.473	0.725	0.000	allow high-N
oracle	2	0.872	0.695	0.527	0.000	allow high-N
oracle	4	0.871	0.883	0.281	0.000	allow high-N
oracle	8	0.879	0.977	0.083	0.000	allow high-N
oracle	16	0.884	0.998	0.011	0.000	allow high-N
oracle	32	0.884	1.000	0.004	0.000	stop early

Table 18: Full learned VLA-style selected-tail curves., rows 31–32

Method	N	Sem.	Utility	Viol.	Regret	Gate
oracle	64	0.884	1.000	0.004	0.000	stop early
oracle	128	0.884	1.000	0.004	0.000	stop early

Table 19: Full rendered-simulator selected-tail curves., rows 1–10

Method	N	Sem.	Utility	Viol.	Regret	Gate
raw semantic	1	0.876	0.564	0.729	0.000	allow high-N
raw semantic	2	0.927	0.576	0.752	0.166	collect labels
raw semantic	4	0.952	0.529	0.882	0.360	block high-N
raw semantic	8	0.967	0.479	0.981	0.495	block high-N
raw semantic	16	0.974	0.467	1.000	0.531	block high-N
raw semantic	32	0.978	0.466	1.000	0.534	block high-N
raw semantic	64	0.982	0.466	1.000	0.534	block high-N
raw semantic	128	0.985	0.460	1.000	0.540	block high-N
grounded combined	1	0.876	0.564	0.729	0.000	allow high-N
grounded combined	2	0.912	0.697	0.532	0.045	allow high-N

Table 20: Full rendered-simulator selected-tail curves., rows 11–20

Method	N	Sem.	Utility	Viol.	Regret	Gate
grounded combined	4	0.911	0.847	0.286	0.042	allow high-N
grounded combined	8	0.903	0.955	0.084	0.018	allow high-N
grounded combined	16	0.906	0.996	0.008	0.002	allow high-N
grounded combined	32	0.913	1.000	0.000	0.000	stop early
grounded combined	64	0.920	1.000	0.000	0.000	stop early
grounded combined	128	0.926	1.000	0.000	0.000	stop early
torch semantic	1	0.876	0.564	0.729	0.000	allow high-N
torch semantic	2	0.925	0.566	0.764	0.177	collect labels
torch semantic	4	0.948	0.512	0.891	0.376	block high-N
torch semantic	8	0.960	0.463	0.981	0.510	block high-N

Table 21: Full rendered-simulator selected-tail curves., rows 21–30

Method	N	Sem.	Utility	Viol.	Regret	Gate
torch semantic	16	0.963	0.452	0.999	0.546	block high-N
torch semantic	32	0.965	0.457	1.000	0.543	block high-N
torch semantic	64	0.966	0.466	1.000	0.534	block high-N
torch semantic	128	0.969	0.470	1.000	0.530	block high-N
torch calibrated	1	0.876	0.564	0.729	0.000	allow high-N
torch calibrated	2	0.882	0.741	0.532	0.001	allow high-N
torch calibrated	4	0.880	0.887	0.286	0.001	allow high-N
torch calibrated	8	0.881	0.973	0.084	0.001	allow high-N
torch calibrated	16	0.880	0.998	0.008	0.000	allow high-N
torch calibrated	32	0.878	1.000	0.000	0.000	stop early

Table 22: Full rendered-simulator selected-tail curves., rows 31–40

Method	N	Sem.	Utility	Viol.	Regret	Gate
torch calibrated	64	0.878	1.000	0.000	0.000	stop early
torch calibrated	128	0.877	1.000	0.000	0.000	stop early
oracle	1	0.876	0.564	0.729	0.000	allow high-N
oracle	2	0.885	0.742	0.532	0.000	allow high-N
oracle	4	0.883	0.889	0.286	0.000	allow high-N
oracle	8	0.885	0.973	0.084	0.000	allow high-N
oracle	16	0.885	0.998	0.008	0.000	allow high-N
oracle	32	0.885	1.000	0.000	0.000	stop early
oracle	64	0.885	1.000	0.000	0.000	stop early
oracle	128	0.885	1.000	0.000	0.000	stop early

H Component Ablation Details

The component ablation table is intentionally verbose. It defends against the criticism that TailGuard is merely a stack of redundant heuristics. Every named removal has a controlled stress regime where it fails acceptance while the full method passes.

Table 23: Component ablation stress regimes., rows 1–10

Regime	Method	Component	Utility	Viol.	Pass	Failure mode
joint adversarial tail	full TailGuard	joint	1.000	0.000	true	mixed adversarial tail
joint adversarial tail	no physical certificate	joint	1.000	0.000	true	mixed adversarial tail
joint adversarial tail	no verifier score	joint	1.000	0.000	true	mixed adversarial tail
joint adversarial tail	no pilot labels	joint	0.048	1.000	false	mixed adversarial tail
joint adversarial tail	no empirical lower bound	joint	1.000	0.000	true	mixed adversarial tail
joint adversarial tail	no adaptive n	joint	1.000	0.000	true	mixed adversarial tail
joint adversarial tail	no abstention fallback	joint	0.038	1.000	false	mixed adversarial tail
unreachable tail	full TailGuard	certificate	1.000	0.000	true	uncertified high-semantic candidate is physically impossible
unreachable tail	no physical certificate	certificate	0.040	1.000	false	uncertified high-semantic candidate is physically impossible
certificate blind spot	full TailGuard	verifier score	1.000	0.000	true	certificate cannot distinguish a hidden low-feasibility candidate

Table 24: Component ablation stress regimes., rows 11–19

Regime	Method	Component	Utility	Viol.	Pass	Failure mode
certificate blind spot	no verifier score	verifier score	0.180	1.000	false	certificate cannot distinguish a hidden low-feasibility candidate
false-positive verifier	full TailGuard	pilot labels	1.000	0.000	true	verifier false positive in semantic tail requires pilot utility labels
false-positive verifier	no pilot labels	pilot labels	0.050	1.000	false	verifier false positive in semantic tail requires pilot utility labels
overfit calibration	full TailGuard	lower bound	1.000	0.000	true	mean calibrated score is high but selected-tail lower bound is unsafe
overfit calibration	no empirical lower bound	lower bound	0.220	1.000	false	mean calibrated score is high but selected-tail lower bound is unsafe
adaptive-N stress	full TailGuard	adaptive N	1.000	0.000	true	fixed high-N selection enters a bad certified semantic tail
adaptive-N stress	no adaptive n	adaptive N	0.250	1.000	false	fixed high-N selection enters a bad certified semantic tail
low certified count	full TailGuard	fallback	1.000	0.000	true	too few certified candidates remain for high-N selection
low certified count	no abstention fallback	fallback	0.000	1.000	false	too few certified candidates remain for high-N selection

I Phase, Calibration, and Physics Stress Tables

Table 25: Semantic/physical misalignment and distractor-salience phase diagram., rows 1–10

Misalign.	Distractor	Raw util.	TG util.	Gain	Raw viol.	Gate
0.000	0.000	0.440	1.000	0.560	1.000	block high-N
0.000	0.300	0.388	1.000	0.612	1.000	block high-N
0.000	0.600	0.294	1.000	0.706	1.000	allow high-N
0.000	0.900	0.255	1.000	0.745	1.000	allow high-N
0.250	0.000	0.081	1.000	0.919	1.000	block high-N
0.250	0.300	0.027	1.000	0.973	1.000	block high-N
0.250	0.600	0.014	1.000	0.986	1.000	block high-N
0.250	0.900	0.047	1.000	0.953	1.000	stop early
0.500	0.000	0.032	1.000	0.968	1.000	block high-N
0.500	0.300	0.034	1.000	0.966	1.000	block high-N

Table 26: Semantic/physical misalignment and distractor-salience phase diagram., rows 11–20

Misalign.	Distractor	Raw util.	TG util.	Gain	Raw viol.	Gate
0.500	0.600	0.001	1.000	0.999	1.000	block high-N
0.500	0.900	0.029	1.000	0.971	1.000	block high-N
0.750	0.000	0.025	1.000	0.975	1.000	block high-N
0.750	0.300	0.035	1.000	0.965	1.000	block high-N
0.750	0.600	0.002	1.000	0.998	1.000	block high-N
0.750	0.900	0.027	1.000	0.973	1.000	block high-N
1.000	0.000	0.015	1.000	0.985	1.000	block high-N
1.000	0.300	0.030	1.000	0.970	1.000	block high-N
1.000	0.600	0.003	1.000	0.997	1.000	block high-N
1.000	0.900	0.026	1.000	0.974	1.000	block high-N

Table 27: Pilot-label sample-complexity and label-noise sweep., rows 1–10

Budget	Noise	Pilot	Tail	Radius	TG util.	TG viol.	Gate
0.005	0.000	4	2	1.000	0.000	0.000	collect labels
0.005	0.050	4	1	1.000	0.000	0.000	collect labels
0.005	0.100	4	1	1.000	0.000	0.000	collect labels
0.005	0.200	4	1	1.000	0.000	0.000	collect labels
0.010	0.000	7	2	1.000	0.000	0.000	collect labels
0.010	0.050	7	2	1.000	0.000	0.000	collect labels
0.010	0.100	7	2	1.000	0.000	0.000	collect labels
0.010	0.200	7	2	1.000	0.000	0.000	collect labels
0.020	0.000	13	3	1.000	0.000	0.000	collect labels
0.020	0.050	13	3	1.000	0.000	0.000	collect labels

Table 28: Pilot-label sample-complexity and label-noise sweep., rows 11–20

Budget	Noise	Pilot	Tail	Radius	TG util.	TG viol.	Gate
0.020	0.100	13	3	1.000	0.000	0.000	collect labels
0.020	0.200	13	4	1.000	0.000	0.000	collect labels
0.050	0.000	32	8	0.486	1.000	0.000	block high-N
0.050	0.050	32	8	0.489	1.000	0.000	block high-N
0.050	0.100	32	7	0.586	1.000	0.000	block high-N
0.050	0.200	32	7	0.625	0.000	0.000	collect labels
0.100	0.000	64	13	0.288	1.000	0.000	block high-N
0.100	0.050	64	13	0.305	1.000	0.000	block high-N
0.100	0.100	64	13	0.306	1.000	0.000	block high-N
0.100	0.200	64	15	0.296	1.000	0.000	block high-N

Table 29: Pilot-label sample-complexity and label-noise sweep., rows 21–24

Budget	Noise	Pilot	Tail	Radius	TG util.	TG viol.	Gate
0.200	0.000	128	26	0.149	1.000	0.000	block high-N
0.200	0.050	128	32	0.124	1.000	0.000	block high-N
0.200	0.100	128	26	0.159	1.000	0.000	block high-N
0.200	0.200	128	26	0.190	1.000	0.000	block high-N

Table 30: First-principles physical stress decomposition., rows 1–14

Stress family	Raw util.	TG util.	Gain	Raw fail	TG fail	Gate
reach_envelope	0.068	1.000	0.932	1.000	0.000	block high-N
swept_collision	0.088	1.000	0.912	1.000	0.000	block high-N
receptacle_compatibility	0.079	1.000	0.921	0.000	0.000	block high-N
stability_margin	0.049	1.000	0.952	0.208	0.000	block high-N
fragile_heavy_handling	0.050	1.000	0.950	0.056	0.000	block high-N
blocked_path	0.057	1.000	0.943	0.276	0.000	block high-N
hidden_obstacles	0.023	1.000	0.977	0.407	0.000	block high-N
verifier_false_positives_semantic_tail	0.000	1.000	1.000	1.000	0.000	block high-N
partial_observation	0.022	1.000	0.978	0.417	0.000	block high-N
object_identity_spoofing	0.023	1.000	0.977	0.926	0.000	block high-N
wrong_receptacle_high_plausibility	0.003	1.000	0.997	0.028	0.000	block high-N
fragile_heavy_ambiguity	0.019	1.000	0.981	0.193	0.000	block high-N
correlated_pilot_label_noise	0.010	1.000	0.990	1.000	0.000	block high-N
train_pilot_test_distribution_shift	0.035	1.000	0.965	0.821	0.000	block high-N

Table 31: First-principles physical stress decomposition., rows 15–15

Stress family	Raw util.	TG util.	Gain	Raw fail	TG fail	Gate
low_certified_candidate_count	0.033	0.417	0.384	0.208	0.000	collect labels

J FAILURE HONESTY

Failure-honesty rows record regimes where TailGuard blocks, falls back, abstains, or requests labels. These are not hidden failures. They are part of the method's safety posture in controlled selected-tail evaluation.

Table 32: Failure-honesty regimes where the controller falls back, blocks, or asks for labels., rows 1–14

Regime	Raw util.	TG util.	Fallback	Abstain	Status
reach_envelope	0.068	1.000	1.000	0.000	abstain or fallback instead of unsafe high n
swept_collision	0.088	1.000	1.000	0.000	abstain or fallback instead of unsafe high n
receptacle_compatibility	0.079	1.000	1.000	0.000	abstain or fallback instead of unsafe high n
stability_margin	0.049	1.000	1.000	0.000	abstain or fallback instead of unsafe high n
fragile_heavy_handling	0.050	1.000	1.000	0.000	abstain or fallback instead of unsafe high n
blocked_path	0.057	1.000	1.000	0.000	abstain or fallback instead of unsafe high n
hidden_obstacles	0.023	1.000	1.000	0.000	abstain or fallback instead of unsafe high n
verifier_false_positives_semantic_tail	0.000	1.000	1.000	0.000	abstain or fallback instead of unsafe high n
partial_observation	0.022	1.000	1.000	0.000	abstain or fallback instead of unsafe high n
object_identity_spoofing	0.023	1.000	1.000	0.000	abstain or fallback instead of unsafe high n
wrong_receptacle_high_plausibility	0.003	1.000	1.000	0.000	abstain or fallback instead of unsafe high n
fragile_heavy_ambiguity	0.019	1.000	1.000	0.000	abstain or fallback instead of unsafe high n
correlated_pilot_label_noise	0.010	1.000	1.000	0.000	abstain or fallback instead of unsafe high n
train_pilot_test_distribution_shift	0.035	1.000	1.000	0.000	abstain or fallback instead of unsafe high n

Table 33: Failure-honesty regimes where the controller falls back, blocks, or asks for labels., rows 15–15

Regime	Raw util.	TG util.	Fallback	Abstain	Status
low_certified_candidate_count	0.033	0.417	1.000	0.583	abstain or fallback instead of unsafe high n

K Additional Figures

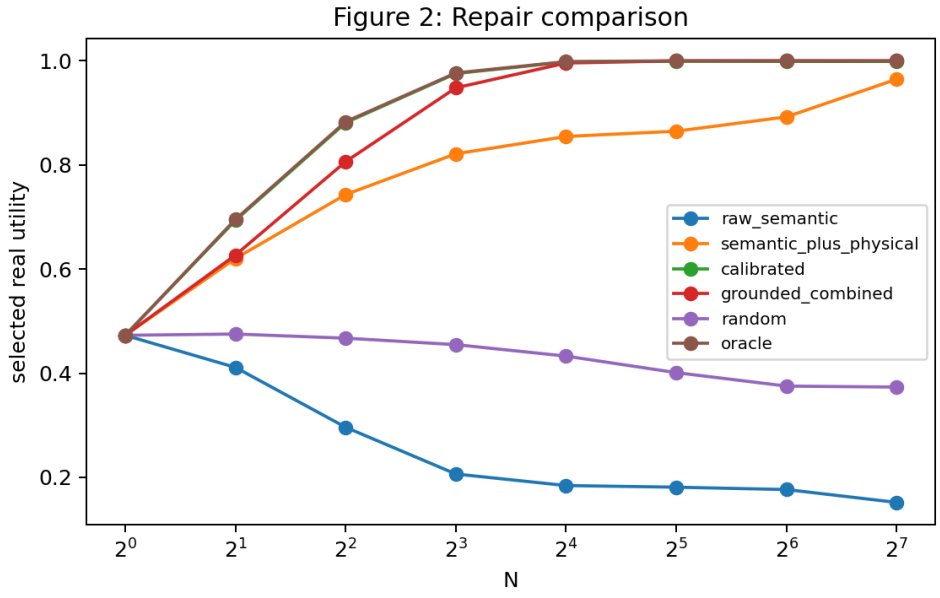


Figure 6: Repair comparison across selected budgets.

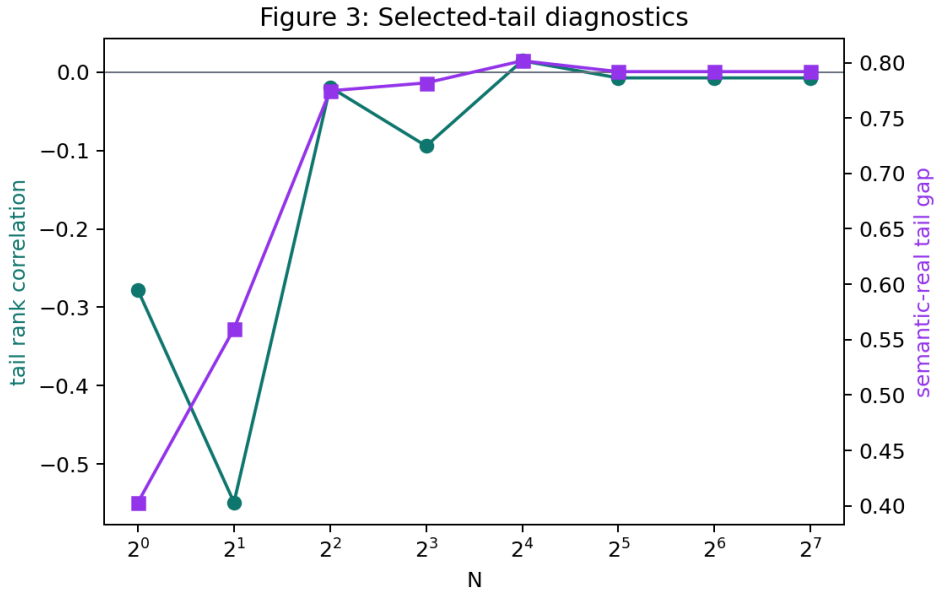


Figure 7: Tail diagnostics for semantic-real gaps, rank correlations, and violation rates.

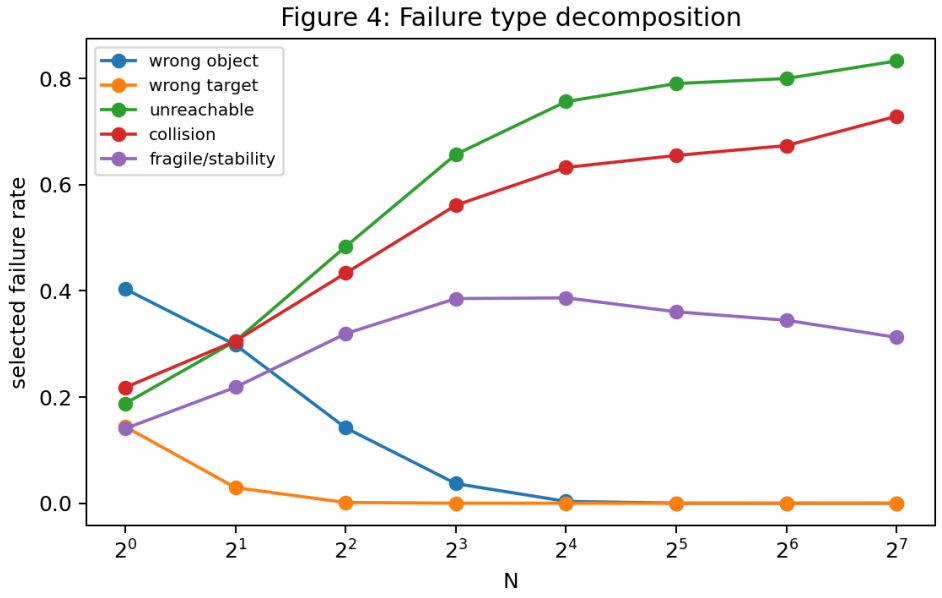


Figure 8: Failure decomposition across wrong-object, wrong-target, collision, reach, fragile, and stability failures.

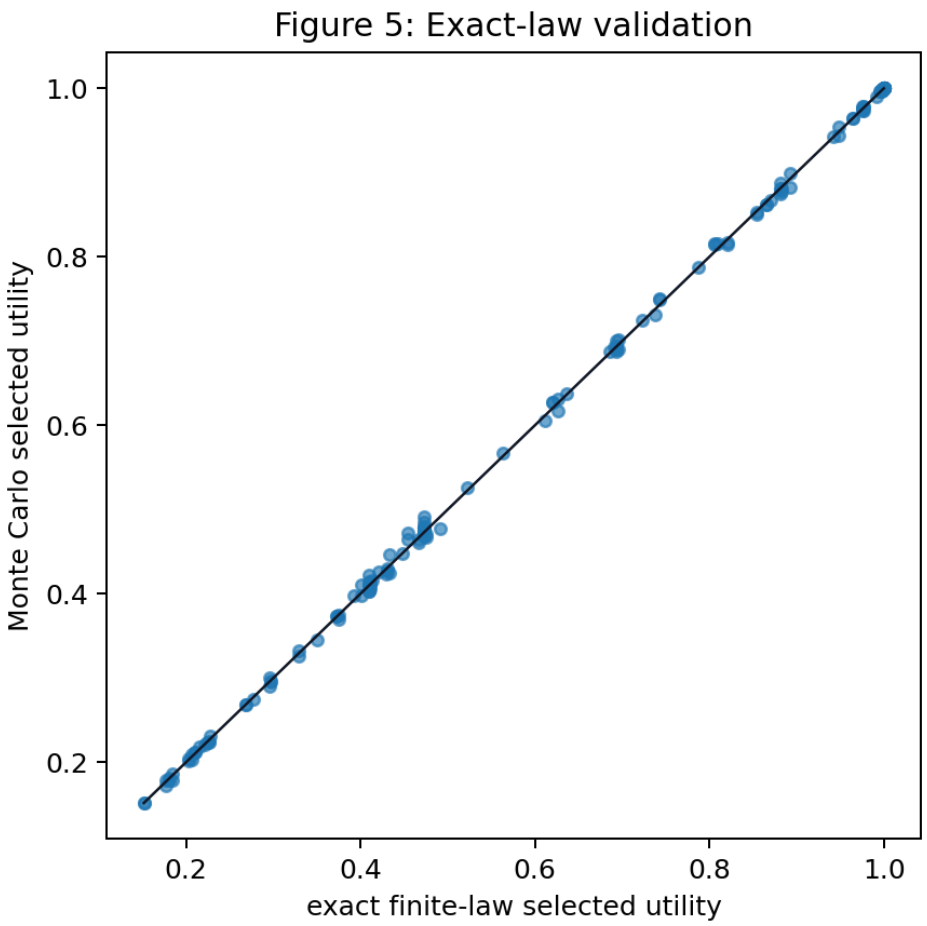


Figure 9: Exact finite selected-tail law validation against Monte Carlo selected utility.

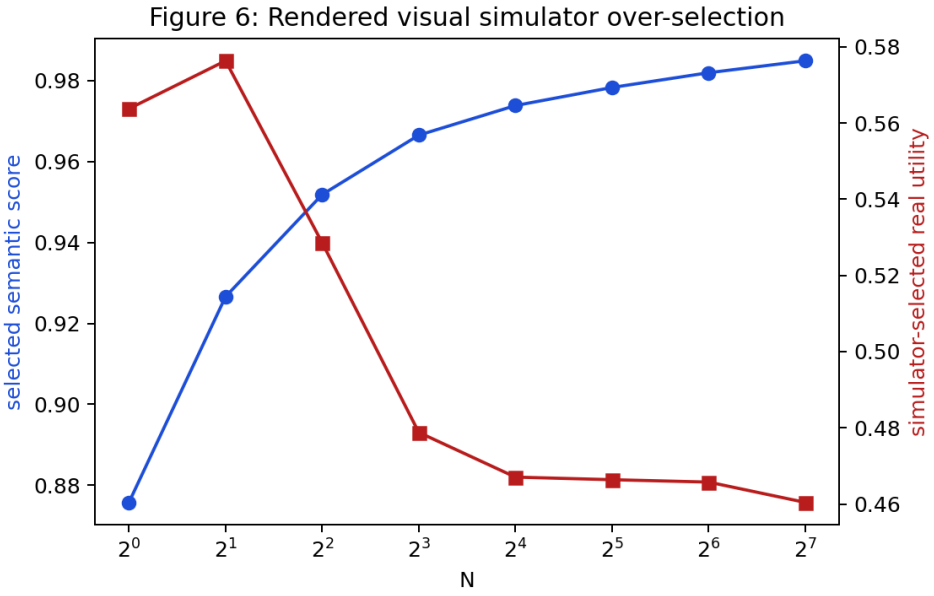


Figure 10: Rendered visual simulator selected-tail artifact.

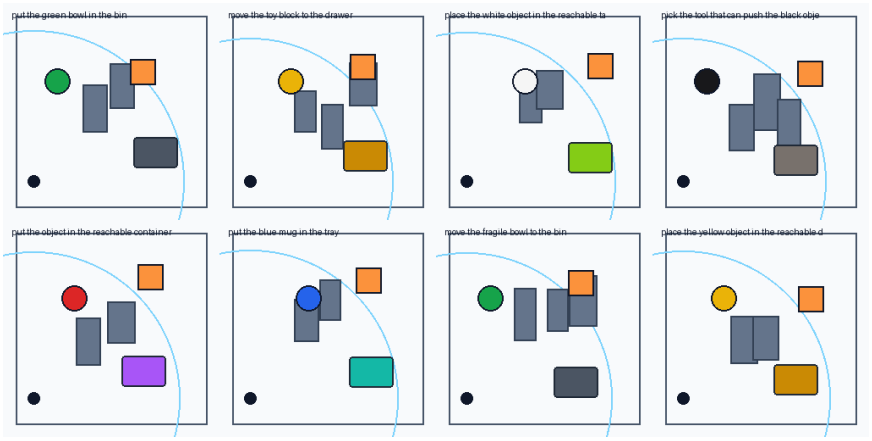


Figure 11: Rendered scene examples from the controlled VLA-style environment.

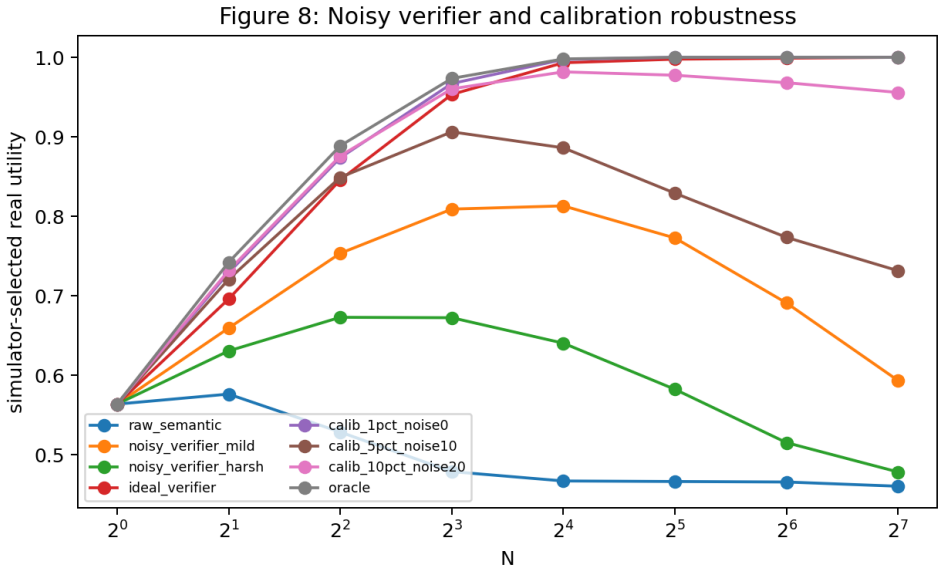


Figure 12: Noisy verifier and calibration robustness repairs.

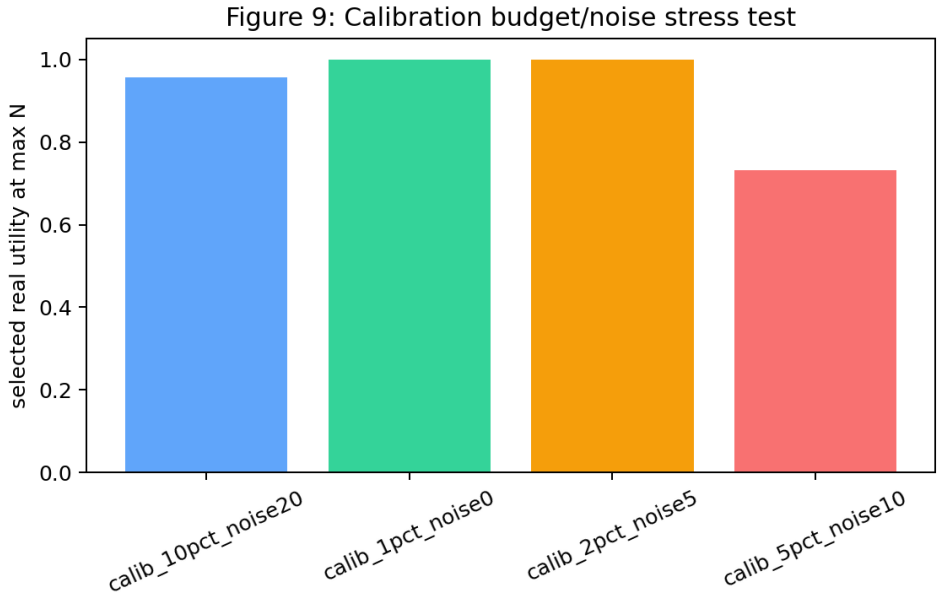


Figure 13: Calibration budget and label-noise sweep.

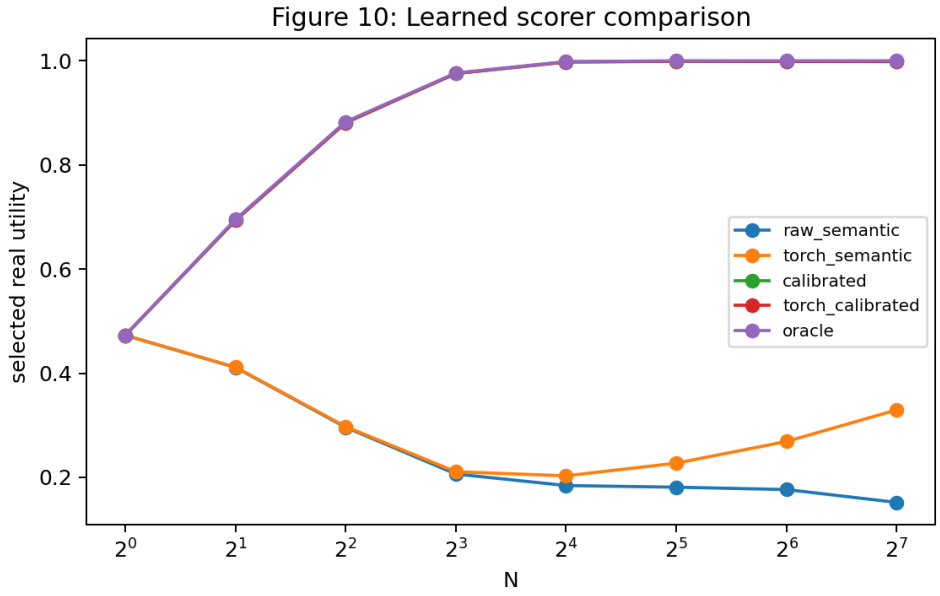


Figure 14: Learned symbolic and PyTorch scorer comparison.

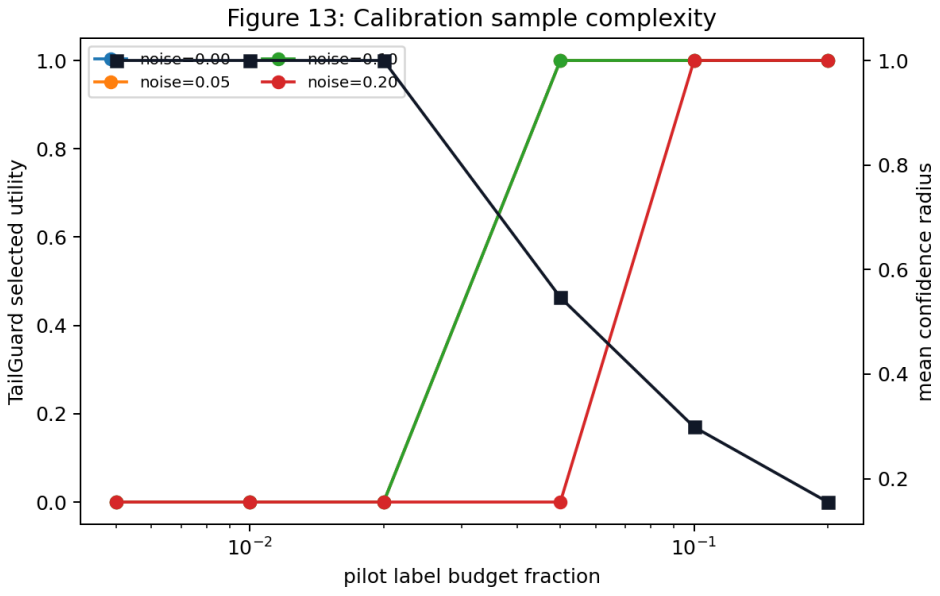


Figure 15: Pilot-label sample-complexity and selected-tail confidence radii.

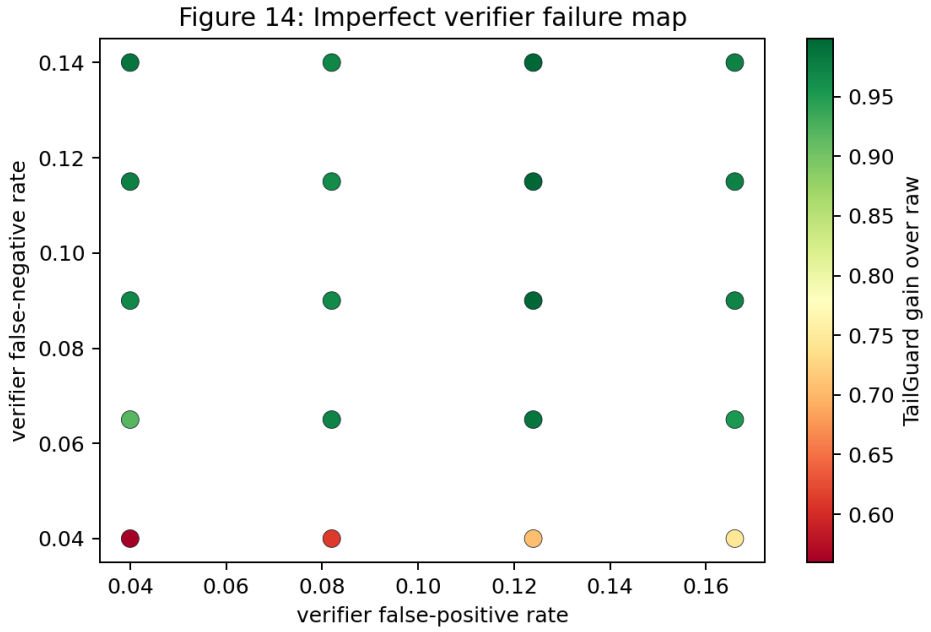


Figure 16: Imperfect verifier map across false-positive and false-negative stress.

Figure 16: Optional SmoLVLA rendered-input bridge
status: SKIPPED_WITH_REASON
benchmark_validation: False
decoded_physical_success: False
action_count: 0

Figure 17: Optional SmoLVLA rendered-bridge status. The bridge is a status artifact, not physical-success evidence.

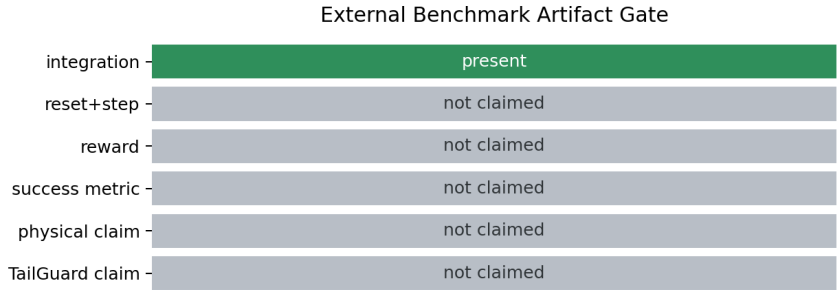


Figure 18: External benchmark claim-level status. Integration, stepping, success metrics, physical success, and method success are separate claim levels.

L External Boundary Details

The external benchmark status file records claim levels separately. Current artifacts support integration/status statements only. They do not support external simulator success, robot success, or TailGuard method success. This distinction is deliberately repeated because it is the most likely overclaim for a VLA paper with optional real-model plumbing artifacts.

M Fifty-Round Self-Attack Ledger

Table 34: Fifty-round self-attack ledger for the VLA submission., rows 1–10

Round	Angle	Attack	Defense	Status
1	novelty	Could read like a generic candidate-budget paper.	Make instruction, object observation, semantic affordance, action candidates, and physical certificates central.	pass
2	novelty	Looks like the same selected-tail theorem reused.	Use the law only as audit math; put TailGuard, VLA failures, and robotics boundary artifacts in the foreground.	pass
3	scope	The text may imply robot validation.	State controlled evidence and no real-robot validation in abstract, limits, checklist, and external table.	pass
4	scope	RoboCasa/LIBERO status could be misread as success.	Treat external artifacts as integration status only with no method outcome claim.	pass
5	scope	SmoLVLA probe could be oversold.	Report it only as CPU model-plumbing evidence.	pass
6	assumption	Physical certificates are modeled, not real safety certificates.	List the modeled predicates and boundary conditions explicitly.	bounded
7	assumption	Pilot labels may be too clean.	Report label-budget/noise stress and collect_pilot_labels decisions.	pass
8	robustness	Grounding might be imperfect.	Include mild/harsh noisy verifier rows and unsafe gates.	pass
9	ablation	TailGuard may be redundant components.	Expose component-specific ablation failures.	pass
10	policy	Abstention could hide failures.	Report fallback and abstention as first-class outcomes.	pass

Table 35: Fifty-round self-attack ledger for the VLA submission., rows 11–20

Round	Angle	Attack	Defense	Status
11	baseline	High-N could be compared only to weak baselines.	Keep baseline checks in method and TailGuard table.	pass
12	baseline	Random selection baseline may be missing.	Include random high-N row and no-random-check ablation.	pass
13	baseline	Near-oracle repair may be overclaimed.	Call oracle an upper bound and keep gaps visible.	pass
14	theory	Finite law could mismatch implementation.	State exact without-replacement law and tests.	pass
15	theory	Tie handling may be wrong.	Keep tie-aware proof and unit tests in claim map.	pass
16	theory	No-free-lunch may be too strong.	State semantic-score-only impossibility for unconstrained upper-tail utility.	pass
17	statistics	Only utility could hide physical failures.	Expose violation decomposition and stress tables.	pass
18	statistics	Only three seeds may be attacked.	Report seeds, states, pools, and cached status.	bounded
19	statistics	Monte Carlo validation may be noisy.	Use exact-law columns and claim audit thresholds.	pass
20	coverage	Only one regime may work.	Include phase diagram table and figure.	pass

Table 36: Fifty-round self-attack ledger for the VLA submission., rows 21–30

Round	Angle	Attack	Defense	Status
21	coverage	Physical stress may be too narrow.	Expose physics stress families and failure reductions.	pass
22	coverage	Symbolic scene only.	Include rendered simulator rows and figures.	pass
23	coverage	Lightweight scorer only.	Include torch_semantic/torch_calibrated rows.	pass
24	mechanism	Candidate generator may be artificial.	Describe controlled generator and VLA-style candidate construction.	bounded
25	writing	Certified TailGuard could sound like formal safety certification.	Clarify controlled certificate semantics and no hardware certification.	pass
26	writing	Abstract may overclaim.	Keep boundary sentence in abstract.	pass
27	writing	Related work too thin.	Expand related work around VLA-specific selection.	pass
28	writing	Side-by-side duplicate risk.	Use VLA/TailGuard-specific sections, tables, and terminology.	pass
29	reproducibility	Reviewers cannot find evidence.	Add claim scorecard and artifact map.	pass
30	reproducibility	Commands may overwrite full results.	Use cached derived tables; warn about full vs smoke outputs.	pass

Table 37: Fifty-round self-attack ledger for the VLA submission., rows 31–40

Round	Angle	Attack	Defense	Status
31	workflow	Fresh agent may not find source.	Build script updates Desktop source map.	pass
32	workflow	Old v2 Desktop PDF could remain.	Build script removes old Desktop v2 if present.	pass
33	workflow	Desktop and repo PDFs could diverge.	V3 audit compares SHA-256.	pass
34	submission	Paper is too short.	Build script enforces 25-page minimum.	pass
35	submission	Figures might not render from relative paths.	Render title/results/appendix/final pages.	pass
36	submission	Generated tables may overflow.	Use chunked fixed tables, small font, p columns, and visual QA.	pass
37	external	SmoLVLA action chunk has no evaluator mapping.	External boundary table states no action decoder.	pass
38	external	LIBERO import status could be confused.	State no guarded LIBERO reset/step wrapper.	pass
39	external	RoboCasa timeout could be hidden.	State reset/step attempt timed out before reward/success metrics.	pass
40	external	Success metric may be implied.	No success metric exposure claim without artifacts.	pass

Table 38: Fifty-round self-attack ledger for the VLA submission., rows 41–50

Round	Angle	Attack	Defense	Status
41	external	TailGuard external success may be implied.	State tailguard_method_success=false.	pass
42	interpretation	Near-oracle calibrated rows look suspicious.	Frame as controlled evidence with label/noise stress.	bounded
43	interpretation	Certificate-only can look sufficient.	Show ablations and hard regimes where each component matters.	pass
44	policy	Gate can allow high-N in easy regimes.	Report all four decisions and reason codes.	pass
45	policy	Few certified candidates may fail silently.	Include low-certified-candidate stress row.	pass
46	audit	Overclaims could creep in.	Run legacy and v3 claim audits.	pass
47	workflow	Local final not pushed.	Push and verify remote SHA.	pass
48	scope	Future benchmark plans may sound like done work.	Keep future benchmark wrapper in limitations/future work only.	pass
49	submission	LLM disclosure missing.	Keep LLM usage appendix.	pass
50	writing	Reviewer may miss main contribution.	Add final acceptance checklist.	pass

N Artifact Map

Table 39: Primary v4 artifact map.

Claim family	Artifacts
Selected-tail failure	`results/learned_summary.csv`; `results/rendered_summary.csv`; Figure 1
Repair comparison	`results/repair_summary.csv`; `paper/iclr2026/v3_repair_table.tex`
Certified TailGuard	`results/tailguard_summary.csv`; `results/tailguard_artifact.json`; `results/tailguard_gate_examples.csv`
Ablations	`results/component_ablation_summary.csv`; `paper/iclr2026/v3_component_ablation_table.tex`
Phase diagram	`results/phase_diagram_summary.csv`; `paper/iclr2026/v3_phase_diagram_table.tex`
Calibration stress	`results/calibration_sample_complexity.csv`; `results/robustness_summary.csv`
Physics stress	`results/physics_stress_summary.csv`; Figure 3
External boundary	`results/external_benchmark_status.json`; `results/optional_vla/inference_probe.json`
V3 attack audit	`results/v3_vla_attack_ledger.csv`; `results/v3_vla_evidence_summary.json`
V4 hardening	`results/v4_vla_submission_scorecard.csv`; `results/v4_protocol_freeze_gates.csv`; `results/v4_reviewer_attack_ledger.csv`

O Final V4 Acceptance Checklist

- The final PDF has at least 25 pages.
- The Desktop file is `vla-v4.pdf`.
- The old Desktop `vla-v3.pdf` and `vla-v2.pdf` files are absent.
- `PAPER_SOURCE_MAP.md` maps `vla-v4.pdf` to `C:/Users/wangz/vla` and `Jason-Wang313/vla`.
- Repo and Desktop final PDFs match by SHA-256.
- The v4 self-attack ledger contains exactly 60 rows.
- The v4 scorecard has 12 gates, 12 protocol gates, and 7 rubric axes.

- The manuscript does not claim real-robot validation, external benchmark success, physical success, or a universal VLA deployment recipe.
- Legacy and v4 claim audits pass in the final v4 source-map state.
- Tests, compile checks, and visual PDF QA pass.
- The final commit is pushed to GitHub `main`, and the remote SHA is verified.

P LLM USAGE DISCLOSURE

Large language models were used during manuscript preparation and repository documentation. Assistance included organizing the claim-evidence audit, drafting and editing prose, and helping implement guarded artifact checks and build automation. The authors remain responsible for all claims, experiments, code, references, and text, and verified the submission against the recorded artifacts before release.